Memory Archive: A Memory-Grounded Training Paradigm for Computer Use Agents

Kartik A.
Independent Researcher
Project Dockyard
kartikswamy2000@gmail.com
DOI: 10.5281/zenodo.20176599

Abstract

A research analysis of the training paradigm enabled by the Memory Archive data collection system, developed as part of Project Dockyard. Memory Archive produces a structured, annotated dataset comprising per-step actuation records, process-level reasoning annotations, visual state triples, and compiled task guides called memories. This data is used across all four stages of the CUA training and deployment lifecycle: pre-training, supervised fine-tuning, post-training reinforcement, and inference-time retrieval. Reasoning annotations are produced by a VLM Reasoning Model as the primary source, with human annotation as an alternative mode. The document covers all four training stages at full technical depth including mathematical formulations, actuation artifact treatment, data construction pipelines, algorithm specifications, hyperparameter guidance, and failure mode analysis. A fifth section covers self-generated memory as an in-training evaluation mechanism.

Key thesis: *The central novelty of this paradigm is* format consistency. *The* `memory.md` *artifact is the same structured object at pre-training, fine-tuning, post-training, and inference time. This eliminates the train-deploy distribution gap that undermines most CUA systems. Additionally, the trained model can generate its own memories at inference time, growing the library continuously and providing a multi-dimensional signal for evaluating training progress.*

1 Background and Problem Statement

1.1 Structural Failures of Standard CUA Training

The dominant CUA training pipeline trains on (screenshot, action) pairs via behavioral cloning followed by outcome-supervised RL. Five structural limitations constrain this approach:

- **Outcome sparsity:** binary task success/failure provides no per-step gradient signal for long-horizon tasks.
- **Intent blindness:** the model learns surface action patterns without causal understanding of task structure.
- **Train-deploy format mismatch:** models trained on screenshot/action pairs are deployed with plain-text prompts — a representational distribution gap at every task boundary.
- **No persistent task knowledge:** every task execution is zero-shot regardless of prior identical task experience.
- **Non-compositional generalisation:** monolithic trajectory learning prevents reliable composition of learned sub-procedures into novel task sequences.

1.2 The Memory Archive Architecture

Memory Archive connects to three tool servers during CUA task execution: Control-Center (actuation events via gRPC stream), The-Eyes (screen capture via HTTP), and a VLM Reasoning Model. The VLM is the primary annotation source, operating in real time. Human annotation is available as an alternative mode for bootstrapping, quality assurance, and edge cases outside VLM competence range.

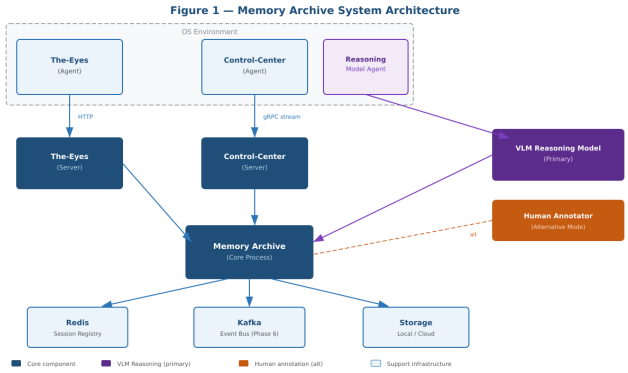

Figure 1: Memory Archive System Architecture. VLM Reasoning Model (purple) is the primary annotation path. Human annotation (orange) is the alternative. Both paths produce the same schema in `reasoning.jsonl`.

2 Data Architecture

Table 1: Inventory of all Memory Archive artifacts, their formats, granularity, and primary use across the four training stages.

Artifact	Format	Granularity	Primary Stage Use
`actuation_commands.json`	JSONL→JSON	Per event	SFT + RL: primary action output target — CommandEvent schema
`cc_commands.json`	JSONL→JSON	Per event	SFT: Control-Center replay format — executable actuation
`raw_input.md` / `converted_input.md`	Markdown	Per event	Pre-training + SFT: action language grounding
`vision/frames/` (before, at, after)	Image (mixed)	Per step (3 frames)	Pre-training + SFT: visual state grounding
`reasoning.jsonl`	JSONL	Per step	SFT + RL: process supervision labels
`memory.md`	Markdown	Per session	All stages: format currency
`metadata.json`	JSON	Per session	Tooling: step manifest and session state
`sync_log.json`	JSON	Per session	Infrastructure: cloud sync tracking

2.1 The Three-Frame Visual Record

For every triggered step, three frames are captured. Before and after frames are stored in the exact format received from The-Eyes — no re-encoding. Keyboard at-frames are also stored in the original

format, unmarked. Mouse at-frames are the exception: because Memory Archive draws the click annotation and then re-encodes the image, marked mouse at-frames are *always* re-encoded as **PNG** regardless of the input format received from The-Eyes. This normalisation is required for lossless annotation preservation — JPEG or WebP re-encoding of an annotated frame introduces compression artefacts around the click marker that can affect ViT patch encoding of the annotation region. The `<|at_img_marked|>` special token therefore always corresponds to a PNG frame; the `<|at_img|>` token for keyboard events preserves the original format. This distinction matters for the special token schema (Section 3.1.5) and for storage integrity.

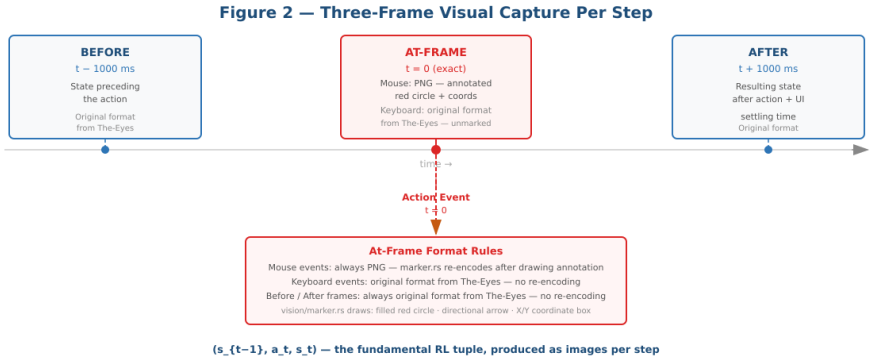

Figure 2: Three-Frame Visual Capture Per Step. Before/After frames and unmarked keyboard at-frames preserve the original format from The-Eyes. Marked mouse at-frames are always re-encoded as PNG after annotation drawing, ensuring lossless click-marker preservation. The complete (s_{t-1}, a_t, s_t) RL tuple is produced as images per step.

2.2 reasoning.jsonl and memory.md

`reasoning.jsonl` contains step-level process supervision labels — produced at VLM scale rather than human scale. `memory.md` compiles these per-step records into a structured procedural document: Overview, Prerequisites, Steps (one subsection per step with reasoning text and at-frame image reference), and Notes & Edge Cases.

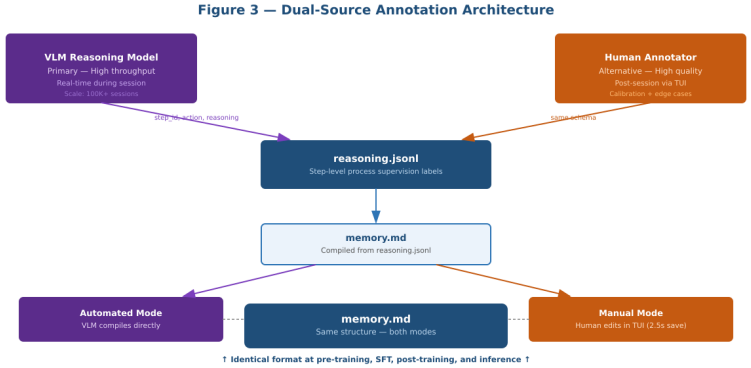

Figure 3: Dual-Source Annotation Architecture. VLM Reasoning Model provides throughput; human annotation provides quality calibration. Both paths produce the same `reasoning.jsonl` schema and compile to the same `memory.md` structure.

3 The Training Pipeline: Full Technical Specification

This section provides full mathematical formulations, data construction specifications, algorithm details, hyperparameter guidance, and failure mode analysis for each training stage. It is intended for ML researchers designing training runs and ML engineers implementing the pipeline.

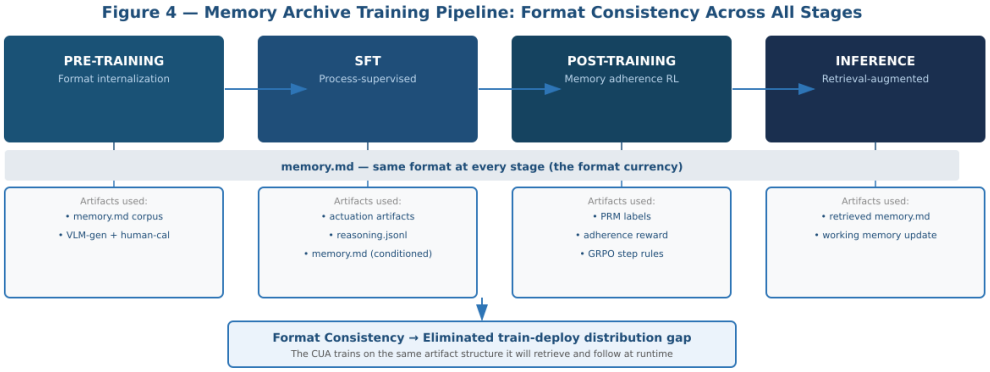

Figure 4: Training Pipeline Overview. `memory.md` threads through all four stages as the shared format currency. Actuation artifacts are primary training targets in SFT alongside reasoning and memory context.

3.1 Pre-Training

3.1.1 Objective

Standard autoregressive language modelling over interleaved image-text sequences composed from Memory Archive artifacts. The objective is format internalization: before any task-specific fine-tuning, the base model must understand what a well-formed memory looks like, how step sections are structured, and how image references relate to action descriptions and actuation commands.

$$\mathcal{L}_{\text{pretrain}} = -\sum_{t} \log p_\theta(x_t \mid x_{<t},\, I_1 \ldots I_k)$$

where $I_1 \ldots I_k$ are the visual token sequences for the before/at/after frames interleaved at their positions in the sequence. Loss is computed over all text tokens. Image tokens do not contribute to the loss.

3.1.2 Data Corpus Construction

Quality Filtering

- Completeness: `reasoning.jsonl` line count equals `metadata.json` `total_steps`. Interrupted sessions excluded.
- Annotation density: skipped step ratio $< 30\%$.
- Image integrity: all at-frames referenced in `metadata.json` must exist with non-zero file size.

Minimum Viable Dataset Scale The paradigm provides no utility below a minimum session count. Analogous systems provide reference points: UI-TARS [6] used $\sim$10B GUI tokens; OSWorld training used hundreds of task trajectories. Estimated minimums for Memory Archive:

- **Phase 1 — Pipeline validation** ($\sim$100 sessions, $\geq$10 distinct tasks): sufficient to validate the data collection pipeline, SFT loss curves, and CommandEvent schema validity on a narrow task set. Not sufficient for generalisation or RL.

- **Phase 2 — Pre-training** ($\sim$1,000 sessions, $\geq$50 distinct tasks): minimum for meaningful format internalization and ScreenSpot accuracy above chance on held-out sessions. Required before SFT is meaningful.
- **Phase 3 — RL training** ($\sim$10,000 GRPO rollouts, $\geq$200 distinct tasks): minimum for GRPO group variance to be non-degenerate across the full task distribution. Below this, the cold-start regime dominates.

These are principled estimates requiring empirical validation (see Section 8). Memory Archive v0.10.0 must collect sessions at scale for the paradigm to be trained; this data collection dependency is the current rate-limiting constraint on the full training pipeline.

Deduplication Near-duplicate sessions collapsed by MinHash LSH on `memory.md` text. Similarity threshold 0.85 — only the highest-annotation-density session retained above this threshold.

Data Mixing Ratios The specific mixing ratios for the pre-training corpus are detailed in Table 2.

Table 2: Pre-training data mixing ratios.

Data Source	Mix Ratio	Rationale
`memory.md` compiled documents	40%	Primary format currency — highest structural richness
`reasoning.jsonl` + image triples	30%	Step-level multimodal alignment — image-action-intent correspondence
Actuation command files (all four)	20%	Action language and CommandEvent schema grounding
General GUI screenshots (optional)	10%	Visual diversity — reduces visual distribution shift at inference

3.1.3 Curriculum Annealing

- **Phase 1** (first 20% of tokens): actuation command files and `converted_input.md` only. The model learns the vocabulary and grammar of GUI actuation events, the CommandEvent schema, and the action taxonomy before seeing reasoning or compiled memories.
- **Phase 2** (next 40% of tokens): add `reasoning.jsonl` + image triples. The model learns to associate visual states with procedural intent at step granularity.
- **Phase 3** (remaining 40% of tokens): add full `memory.md` documents. The model learns the compiled procedural format it will consume at inference and produce during self-evaluation.

3.1.4 Architecture Requirements

- Variable-length multi-image input: a session with N steps requires $3N$ images. Models with fixed image slots are unsuitable.
- Long context: the minimum viable context window depends on the backbone's tokens-per-image at native display resolution. For a fixed-resolution ViT-L/14 at 224×224: 196 patch tokens per image × 3 frames × 50 steps + 4,000 text tokens = 33,400 tokens — already above a 32K window before any memory.md text. For dynamic-resolution backbones (InternVL2, Qwen2-VL) encoding 1080p screenshots: $\sim$3,600 tokens per frame, giving $540{,}000+$ tokens at 50 steps. The context budget formula is:

$$C_{\min} = N_{\text{steps}} \times 3 \times T_{\text{img}} + T_{\text{text}} + T_{\text{memory}}$$

and the maximum session length supportable by a backbone with context limit C_{model} is:

$$N_{\max} = \left\lfloor \frac{C_{\text{model}} - T_{\text{text}} - T_{\text{memory}}}{3 \times T_{\text{img}}} \right\rfloor$$

For a 128K context model at 256 tokens/image: $N_{\max} \approx 161$ steps. **The minimum recommended context window is 128K–256K depending on backbone.** Models with 32K

context are not viable for sessions longer than a few steps at dynamic resolution. Backbone selection must be the first architecture decision, as it sets T_{img} and therefore all session length limits. TurboQuant (Section 7) extends the *effective* context budget by $4.6\times$ within the same hardware, making the 128K minimum more accessible.

- Dynamic resolution: The-Eyes returns images in native display resolution. Fixed-resolution ViT patch encoding discards spatial information required for click coordinate grounding.
- Causal mask over interleaved sequences: image tokens at position p must attend to all prior text and image tokens.

3.1.5 Special Token Schema

These tokens delimit structural components of `memory.md`, actuation records, and frame types. The at-frame token is split by event type because marked mouse at-frames (PNG, annotated) carry different information than unmarked keyboard at-frames (original format, clean screenshot). The after-frame token is required — it carries the outcome confirmation that context-aware reasoning must reference.

```
<|memory_start|>       -- opens a retrieved memory.md document
<|memory_end|>         -- closes a retrieved memory.md document
<|step_start|>         -- opens a step section (### Step N)
<|reasoning|>          -- precedes the reasoning text for a step
<|action|>             -- precedes the CommandEvent JSON for a step
<|converted_action|>   -- precedes the human-readable converted action
<|before_img|>         -- precedes before-frame image tokens (original format)
<|at_img_marked|>      -- precedes mouse at-frame tokens (annotated with click-
    marker)
<|at_img|>             -- precedes keyboard at-frame tokens (original format,
    unmarked)
<|after_img|>          -- precedes after-frame image tokens (original format)
```

3.2 Supervised Fine-Tuning

3.2.1 Actuation as a First-Class Training Target

The actuation artifacts are not supporting metadata — they are the ground truth of what the CUA did at the tool level and must be treated as primary output targets in training alongside reasoning and memory context. Four artifacts contribute directly.

actuation_commands.json — CommandEvent Schema Each entry is a full CommandEvent JSON object per step. This is the model's primary action output target. The model must learn to produce valid CommandEvent JSON at inference time because this is what gets sent to Control-Center to execute actions in the OS environment.

The schema is a **discriminated union** on `action_type`. Mouse events and keyboard events carry mutually exclusive field sets; generating a field from the wrong branch is a schema error:

```
// Mouse event --- mouse_x, mouse_y always required; key/text fields absent
{
  "action_type":    "mouse",
  "action_subtype": "left" | "right" | "double",
  "mouse_x":        integer,   // pixel x-coordinate
  "mouse_y":        integer,   // pixel y-coordinate
  "success":        boolean,
  "timestamp":      ISO 8601 string,
  "agent_id":       string,
  "session_id":     string
}

// Keyboard event --- key_combo/text_input present; mouse coords absent
{
  "action_type":    "keyboard",
  "action_subtype": "press" | "type",
  "key_combo":      string,    // e.g. "ctrl+c", "enter"  [press events]
```

```
  "text_input":     string,    // freeform typed text      [type events]
  "success":        boolean,
  "timestamp":      ISO 8601 string,
  "agent_id":       string,
  "session_id":     string
}
```

Schema validation at inference runs two checks: (1) JSON parseability; (2) conditional field compliance — a generated action where `action_type="keyboard"` but `mouse_x` is non-null is a schema error and triggers re-sampling. The CommandEvent schema validity metric ($> 99\%$) encompasses both checks. The special tokens `<|at_img_marked|>` (mouse) and `<|at_img|>` (keyboard) provide the model a structural signal about which field set is active before it begins generating the JSON.

cc_commands.json — Control-Center Replay Format The replay format is directly executable by the Control-Center actuation layer. A model that learns to produce valid `cc_commands.json` entries can drive the CUA tool directly without an intermediate translation layer.

converted_input.md — The Bridge Artifact Each line is the human-readable conversion of one actuation event. This is the text that appears in the step sections of `memory.md` and what the model reads when consuming a memory at inference. The model must learn to generate both the machine-readable CommandEvent and the human-readable description — `converted_input.md` is the bridge between the two representations.

raw_input.md — Failure Awareness Raw format as emitted by Control-Center, including `[FAILED]` prefixes for failed commands. Training on `raw_input.md` teaches the model to recognise failed actuation patterns and to understand that failure events produce no corresponding images.

3.2.2 Training Objective: Formulation B — Memory-Conditioned SFT

SFT uses Formulation B: the model is trained with a retrieved `memory.md` in context, producing both reasoning and action for each step. This closes the train-deploy gap — the model learns to read and follow a memory at training time, not just at inference time.

Critically, the `memory.md` in context must be retrieved from a *different* session of the same or similar task — not compiled from the session being trained on. Using the same-session memory is a circular oracle: the model reads a perfect summary of the ground truth labels it is also being supervised to produce, which does not generalise to inference conditions where only an imperfect prior is available. Cross-session retrieval uses the same bi-encoder described in Section 3.4.2. For cold-start tasks with no cross-session candidate, a partial memory is used: the first k steps of the closest available memory are included with the remainder masked. Additionally, 10–20% of retrieved memory steps are randomly substituted with plausible-but-incorrect alternatives, forcing the model to learn to detect and override memory errors rather than reproduce them verbatim.

$$\mathcal{L}_{\text{SFT}} = -\sum_{t} w_t \cdot \log p_\theta\big(x_t \mid x_{<t},\, \mathcal{M}^{\neq},\, I_{\text{before},t}\big)$$

where $\mathcal{M}^{\neq}$ denotes a cross-session memory retrieved from a different execution of the same task; $I_{\text{before},t}$ is the before-frame for step t; and w_t is the per-token loss weight. The at_t and after_t frames appear in the sequence *after* action_t (see Section 3.2.3) and therefore do not condition reasoning or action generation — they serve as the visual consequence record that the model attends to when processing step $t+1$.

3.2.3 Per-Step Sequence Order and Multi-Image Attention Masking

Each step t in the training sequence follows this strict ordering:

$$\text{before}_t \;\rightarrow\; \text{reasoning}_t \;\rightarrow\; \text{action}_t \;\rightarrow\; \text{at}_t \;\rightarrow\; \text{after}_t \;\rightarrow\; [\text{step } t{+}1 \text{ begins}]$$

This ordering reflects causal reality: the mouse at-frame is a consequence of the action — it records *where the click landed* after execution — and is therefore unavailable when the model must generate

its reasoning and action. Placing at_t after action_t in training eliminates the train-inference mismatch that would otherwise arise: at inference time, the action executes first, and the at-frame is then captured and appended to the session context before step $t+1$ begins. The after-frame closes each step as visual confirmation that the action produced the intended state.

Attention Mask Rules

- **before**$_t$ tokens attend to: all tokens at positions $<$ before_t (prior steps' text and images, system prompt, `memory.md`).
- Reasoning tokens at step t attend to: all tokens at positions $\leq$ before_t — reasoning is generated from the before-frame and prior context only, not from at_t.
- action_t tokens attend to: all prior tokens including reasoning_t for the same step.
- **at**$_t$ tokens attend to: all tokens at positions $\leq$ action_t (the action record that produced the click, for associating spatial annotation with the command).
- **after**$_t$ tokens attend to: all prior tokens including at_t and action_t — the visual consequence is associated with both the click location and the command.
- **before**$_{t+1}$ tokens attend to: all prior tokens including after_t — the model carries the visual outcome of step t into its context when beginning step $t+1$.

At inference time, after each action executes: (1) Control-Center returns the at-frame (click-annotated for mouse events); (2) The-Eyes captures the after-frame at $t + 1000$ms; (3) both are appended to the session context in the same $\text{at}_t \rightarrow \text{after}_t$ order before reasoning for step $t+1$ begins. This closes the loop correctly without any temporal oracle.

Cross-step image attention is fully enabled — the model at step t can reference images from all prior steps.

3.2.4 LoRA Configuration

```
Target modules: q_proj, k_proj, v_proj, o_proj (attention)
                gate_proj, up_proj, down_proj (MLP)
rank (r):        64
alpha:           128   (effective scale = alpha/r = 2.0)
dropout:         0.05
ViT encoder:     frozen during early SFT (first 30% of training steps)
                 unfrozen (full fine-tuning) with lr_vit = 1e-5 during late SFT
                 --- unfreezing improves click coordinate grounding once the
                   backbone has stabilised on the actuation schema
LLM backbone:    lr_llm = 2e-5
Scheduler:       cosine with warmup (5% of steps)
Batch size:      per-device 1 session, grad_accum=16
```

Note: full ViT fine-tuning in the late SFT phase adds gradient and optimiser state memory for the ViT encoder (≈2.4 GB at FP32 for ViT-L/14 with 307M parameters). If GPU memory is constrained, apply LoRA to the ViT's attention projection layers instead (`q`, `v` projections, rank 16–32) as a memory-efficient alternative to full fine-tuning.

3.2.5 Training Example Construction

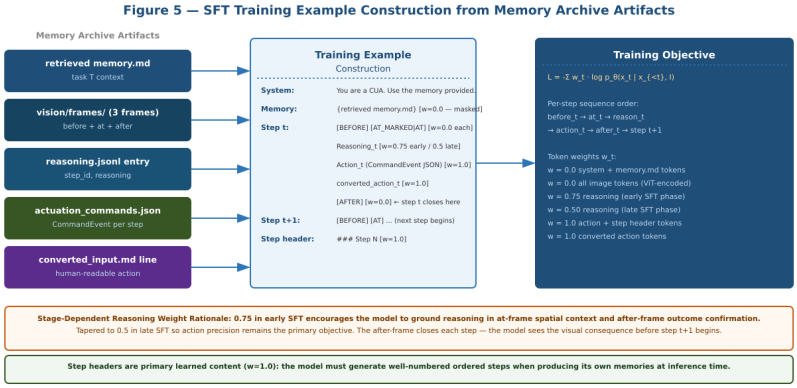

Figure 5: SFT Training Example Construction. All Memory Archive artifacts are assembled into a single multi-step training sequence. Actuation CommandEvent JSON and step headers are full-weight targets ($w = 1.0$). Reasoning uses stage-dependent weighting (0.75 early, 0.5 late). Memory tokens are masked entirely ($w = 0.0$).

3.2.6 Per-Token Loss Weighting

Table 3: Per-token loss weights for SFT (Formulation B).

Token type	**Weight** w_t	**Rationale**
System prompt tokens	0.0	Infrastructure — model reads but is not supervised on prompt wording.
`memory.md` tokens ($\mathcal{M}$)	0.0	Masking prevents learning to reproduce memories verbatim.
Before-frame image tokens	0.0	Encoded by ViT, not generated autoregressively.
At-frame image tokens	0.0	Encoded by ViT. Carries spatial grounding for mouse events.
After-frame image tokens	0.0	Encoded by ViT. Visual confirmation of step completion.
Reasoning tokens (early SFT 30%)	0.75	Higher weight while model learns to ground reasoning spatially.
Reasoning tokens (late SFT 70%)	0.50	Tapered as model already reasons correctly.
CommandEvent action tokens	1.0	Primary CUA objective — precise actuation prediction.
Step header tokens (`### Step N`)	1.0	Model must generate well-numbered, ordered steps.
`converted_action` tokens	1.0	Human-readable action description — primary content in `memory.md`.

3.2.7 Evaluation Metrics for SFT

Table 4: SFT evaluation metrics, definitions, and targets.

Metric	What it measures	Target
ScreenSpot accuracy	Element grounding: correct UI element clicked	$> 70\%$ on held-out sessions
OSWorld task success rate [10]	End-to-end task completion in live OS environment	$> 35\%$ post-SFT, pre-RL
CommandEvent schema validity	% of generated actions parseable to valid CommandEvent, including conditional field compliance	$> 99\%$
Memory adherence rate	% of steps where agent action matches memory procedure	$> 60\%$ on in-distribution tasks
Memory deviation rate	% of steps where model correctly overrides an incorrect step in cross-session retrieved memory	$> 40\%$ (guards against circular oracle failure)
Reasoning context score	Entity overlap between reasoning text and at+after frame content	> 0.75 average over steps

3.3 Post-Training — Memory Adherence RL

3.3.1 The Process Reward Model

The PRM is a step-level quality scorer that transforms `reasoning.jsonl` annotations into a continuous quality signal for the GRPO training loop [4; 5]. Without the PRM, the three-component reward function operates purely on action outputs and visual outcomes — it cannot assess whether the model's intermediate reasoning correctly interprets the task context.

Architecture The PRM is a fine-tuned VLM-scale scorer that takes as input the three-frame visual context $(\text{before}_t, \text{at}_t, \text{after}_t)$ concatenated with the reasoning_t text and outputs a scalar $q_t \in [0, 1]$ representing step-level reasoning quality.

Training Objective

$$\mathcal{L}_{\text{PRM}} = -\mathbb{E}_{(x,y)\sim\mathcal{D}}[y \cdot \log q(x) + (1-y) \cdot \log(1-q(x))]$$

where $x = (\text{before}_t, \text{at}_t, \text{after}_t, \text{reasoning}_t)$ and $y \in \{0, 1\}$ is the quality label. $y = 1$ for human-approved reasoning, $y = 0$ for flagged VLM reasoning or hard negatives.

Human Calibration VLM-generated reasoning annotations carry systematic biases: they tend to overstate action certainty, underspecify visual grounding, and occasionally confabulate UI element names. Human-annotated sessions serve as the calibration anchor. If the distribution of PRM scores assigned to human reasoning entries has mean < 0.80, the PRM has learned to penalise human-quality reasoning and must be recalibrated. Recalibration applies stratified correction factors separately for mouse-action steps and keyboard-action steps, since VLM annotation biases differ systematically between these: $\text{calibration\_weight}_c = 0.80 \,/\, \text{mean}(q(\text{human\_entries}_c))$ for $c \in \{\text{mouse}, \text{keyboard}\}$.

Decoupling PRM Training from the RL Cycle The PRM faces a circular dependency if trained on VLM-annotated sessions: VLM annotation quality is what RL aims to improve, so using VLM annotations as PRM supervision trains a critic on the same corrupted signal as the policy. To break this loop: (1) the PRM is trained *exclusively* on human-annotated sessions before the RL phase begins; (2) it is deployed as a frozen critic during RL Phase 1; (3) it is retrained once per major training cycle using newly human-reviewed sessions from the previous cycle. A minimum human annotation budget of several hundred sessions (targeting PRM AUC > 0.80 on a held-out human-annotated validation set) is required before RL training begins. Similarly, the InfoNCE training data for the reasoning encoder (R_{align}) must be drawn from human-annotated sessions only during the initial training; VLM-annotated sessions may be added in subsequent cycles after the encoder has stabilised.

Integration with the Reward Function The PRM score enters R_{align} as an additive term:

$$R^{\text{PRM}}_{\text{align}}(\tau, \mathcal{M}) = (1 - \lambda) \cdot R_{\text{align}}(\tau, \mathcal{M}) + \lambda \cdot \frac{1}{N} \sum_{t} q_t^{\text{PRM}}, \quad \lambda = 0.2$$

The PRM term detects trajectories that are well-aligned to the memory text in embedding space while still being frame-blind — the model generates reasoning that describes the memory procedure without verifying against the actual visual context.

Update Frequency The PRM is held fixed as a scoring function during RL training. It is retrained once per major training cycle using newly human-reviewed sessions from the previous cycle.

3.3.2 Algorithm: GRPO for CUA

Group Relative Policy Optimization (GRPO, [7]) is used rather than PPO because it eliminates the need for a separate value network. For the CUA context, the primary memory pressure is the KV cache: at FP16, a 50-step session holding 150+ image token encodings occupies ≈70.7 MB per session (Section 7.1). Eliminating a duplicate critic backbone that would hold an identical copy of these image encodings avoids maintaining a second KV cache of the same size — the principal motivation for GRPO over PPO in this setting.

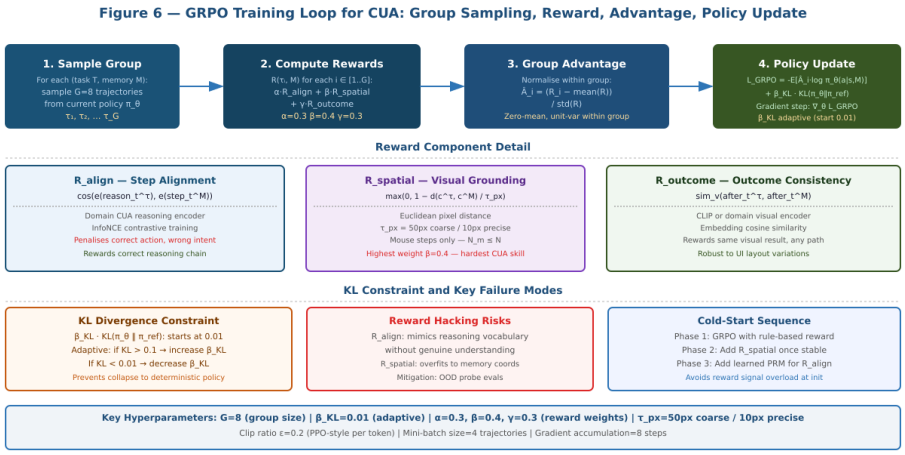

Figure 6: GRPO Training Loop for CUA. For each (task, memory) pair, G=8 trajectories are sampled. Rewards are computed per trajectory, group-relative advantage normalises them, and the policy gradient update is applied with adaptive KL regularisation.

Compute Cost and Infrastructure Requirements Unlike RL for mathematical reasoning where $G = 8$ samples cost microseconds, CUA RL sampling requires live OS instances. Each step involves action execution, UI settling time (≈1,000ms), and screenshot capture. Estimated wall-clock time per trajectory: $N_{\text{steps}} \times$ 3–5s ≈ 60–100s for a 20-step task. At $G = 8$: approximately 8–14 minutes per (task, memory) pair per RL iteration if executed sequentially. **Parallel execution across G OS instances is mandatory.** The minimum infrastructure requirement for viable RL training:

- $G = 8$ parallel OS sandbox instances per training worker (QEMU/KVM or OSWorld-compatible VMs).
- Simulation environments (OSWorld [10], WindowsAgentArena [1]) strongly preferred over live OS for RL rollouts; periodic live evaluation for distribution check.
- Group size $G = 4$ during cold-start warm-up (Section 7.1) halves infrastructure load during the high-uncertainty early phase; G is restored to 8 once policy variance stabilises.
- A dedicated compute requirements section for any deployment should state: number of OS instances, GPU VRAM per worker, and estimated total RL training hours for the target task set.

3.3.3 The Three-Component Reward Function

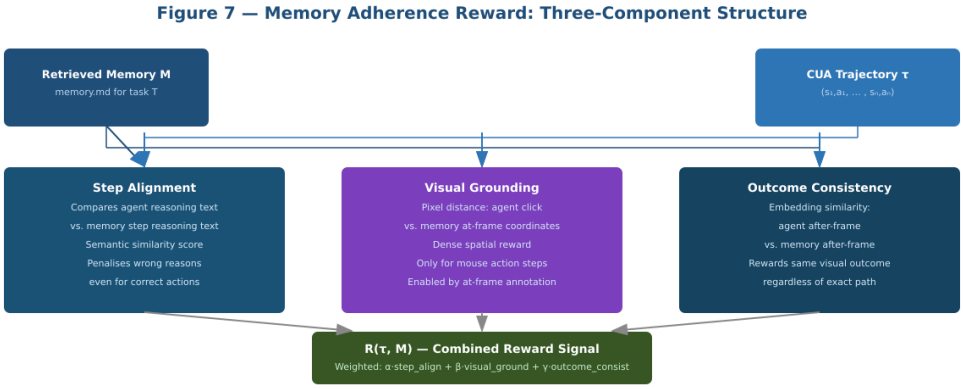

Figure 7: Memory Adherence Reward: Three-Component Structure. Step Alignment, Visual Grounding, and Outcome Consistency are combined as $R(\tau, \mathcal{M}) = \alpha \cdot R_{\text{align}} + \beta \cdot R_{\text{spatial}} + \gamma \cdot R_{\text{outcome}}$ with weights 0.3, 0.4, 0.3.

R_{align} — Step Alignment ($\alpha = 0.3$)

$$R_{\text{align}}(\tau, \mathcal{M}) = \frac{1}{N} \sum_{t=1}^{N} \cos\text{-sim}\big(e(\text{reason}_t^{\tau}),\, e(\text{step}_t^{\mathcal{M}})\big)$$

where $e(\cdot)$ is a domain-specific CUA reasoning encoder trained via InfoNCE contrastive loss. Positive pairs: $(\text{reasoning}_t^{\tau}, \text{step}_t^{\mathcal{M}})$ from the same session. Negative pairs: step text from different sessions for the same action type. Off-the-shelf text embeddings are insufficient — they cannot distinguish between "click submit to confirm" and "click cancel to discard" which have similar surface form but opposite intent.

R_{spatial} — Visual Grounding ($\beta = 0.4$)

$$R_{\text{spatial}}(\tau, \mathcal{M}) = \frac{1}{N_m} \sum_{t \in \text{mouse\_steps}} \max\left(0,\; 1 - \frac{d\big(c_t^{\tau},\, c_t^{\mathcal{M}}\big)}{\tau_{\text{px}}}\right)$$

where $d(\cdot)$ is Euclidean pixel distance, $c_t^{\mathcal{M}}$ is the click coordinate stored in the at-frame annotation of $\mathcal{M}$, $\tau_{\text{px}} = 50$px coarse / 10px precise. Gets the highest weight ($\beta = 0.4$) because spatial precision is the hardest CUA skill to acquire from language supervision alone. Failure mode: overfitting to memory pixel coordinates rather than learning to identify the correct UI element. Mitigation: random affine augmentation on at-frames during training.

R_{outcome} — Outcome Consistency ($\gamma = 0.3$)

$$R_{\text{outcome}}(\tau, \mathcal{M}) = \frac{1}{N} \sum_{t=1}^{N} \text{sim}_v\big(\text{after}_t^{\tau},\; \text{after}_t^{\mathcal{M}}\big)$$

where $\text{sim}_v(\cdot)$ is cosine similarity in a visual encoder embedding space. Computed at embedding level, not pixel level — robust to temporal jitter, font rendering differences, and cursor position. Rewards trajectories that produce the same visual outcomes as the demonstration regardless of exact action sequence.

3.3.4 GRPO Policy Update

$$\mathcal{L}_{\text{GRPO}} = -\mathbb{E}_{\tau_i \sim \pi_\theta}\left[\hat{A}_i \cdot \sum_t \log \pi_\theta(a_t \mid s_t, \mathcal{M})\right] + \beta_{\text{KL}} \cdot \text{KL}(\pi_\theta \| \pi_{\text{ref}})$$

where $\hat{A}_i = (R_i - \text{mean}(R))/\text{std}(R)$ is the group-relative advantage. Group size $G = 8$. PPO-style clipping: clip$(r_t, 1-\varepsilon, 1+\varepsilon)$ with $\varepsilon = 0.2$. β_{KL} adaptive: multiply by 1.5 if KL > 0.1, multiply by 0.8 if KL < 0.01.

3.3.5 Cold-Start Protocol

GRPO's group-relative advantage normalisation requires meaningful variance across $G = 8$ trajectories. The cold-start problem arises when the initial policy generates trajectories that all earn $R \approx 0$, making std$(R) \approx 0$ and rendering the normalised advantage $\hat{A}_i$ numerically degenerate.

Minimum SFT Checkpoint Quality RL training must not begin before the policy achieves: OSWorld [10] task success rate $> 35\%$; Memory adherence rate $> 60\%$; CommandEvent schema validity $> 99\%$.

Rule-Based Reward Warm-Up The first 10% of RL training steps use a simplified reward: only R_{align} is active (weight $\alpha = 1.0$); spatial and outcome components are disabled. Positive reward threshold lowered: any trajectory where step-alignment cosine similarity > 0.3 earns non-trivial R_{warmup}. After warm-up, the full three-component reward is activated with standard weights $\alpha = 0.3$, $\beta = 0.4$, $\gamma = 0.3$.

Task Curriculum for RL Single-step tasks are used exclusively for the first 10% of RL training steps. Multi-step tasks are introduced in order of increasing step count thereafter, reaching the full task distribution at 20% of training steps.

Group Size Annealing During warm-up, group size is reduced to $G = 4$. G is linearly incremented back to 8 over the subsequent 10% of training steps. Transition criterion: G is incremented by 1 each time fewer than 5% of task groups in a training batch exhibit std$(R) < 0.01$.

3.3.6 RL Training Data Construction

Post-training RL requires a distinct corpus governed by stricter constraints than pre-training. Session eligibility requires: complete annotation (skipped step ratio $< 10\%$); minimum annotation quality (human-annotated or VLM-annotated with consistency score above the threshold defined in Section 6.3); compiled and reviewed `memory.md`; and task executability in an available OS environment. GRPO group construction requires $G = 8$ distinct trajectories per (task, memory) pair. Sessions with fewer than three steps are eligible only during warm-up. Generalisation sessions — tasks where the policy succeeded via the new memory creation path — enter the RL corpus after their memory passes `pending_review`. Sessions are sampled with probability proportional to annotation quality score, capped at five sessions per (task, `os_type`) pair. OS type balance is enforced with a floor of 20% for any single OS type.

3.4 Inference-Time Retrieval

3.4.1 Retrieval Pipeline Architecture

At inference time the CUA queries a memory library for the most relevant `memory.md` and uses it as context for execution. The pipeline has two stages: a fast approximate nearest-neighbour search for candidate generation, followed by a cross-encoder re-ranker for final selection. An OS/version pre-filter runs before the ANN search.

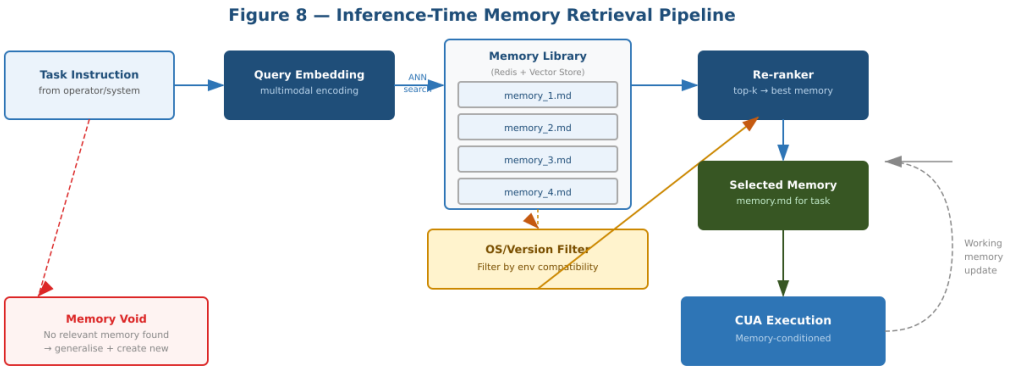

Figure 8: Inference-Time Memory Retrieval Pipeline. The OS/version filter prevents stale memories from being retrieved. The working memory update loop triggers re-retrieval or new memory creation on execution deviation.

3.4.2 Bi-Encoder and HNSW Index

Embedding Model Training

$$\mathcal{L}_{\text{contrastive}} = -\log \frac{\exp(\text{sim}(q, m^+)\,/\,\tau)}{\sum_{m \in \mathcal{B}} \exp(\text{sim}(q, m)\,/\,\tau)}$$

Standard InfoNCE loss. Hard negative mining: memories from the same task category but different OS version or application version. Architecture: dual BERT-style encoders with tied weights, 1024-dim L2-normalised output vectors.

```
M = 32, ef_construction = 200, ef_search = 100
Recall@10 > 95% at 100K memories, latency ~3ms
Sharded by OS type: LINUX / WINDOWS / MACOS
Supports incremental addition without full re-indexing
```

3.4.3 Cross-Encoder Re-Ranker

Concatenates [query — `memory.md`] for full cross-attention. Trained on (query, memory, relevant=0/1) triples using pointwise binary cross-entropy. Returns top-3 for beam search execution or top-1 for greedy. Total latency ≈80ms for 50 candidates in a parallel batch forward pass.

3.4.4 Working Memory Update and New Memory Creation

Once execution begins, deviation from the retrieved memory is tracked per step using the after-frame — the visual state after that specific actuation step has completed and the UI has settled (t + 1000ms).

$$\text{deviation\_score}(t) = 1 - \text{sim}_v\left(\text{after\_frame}_t^{\tau},\ \text{after\_frame}_t^{\mathcal{M}}\right)$$

If $\text{deviation\_score}(t) > 0.4$ for 3 consecutive steps, one of two paths is taken (note: 0.4 is a *calibration parameter* requiring held-out validation against sessions where the correct re-retrieval point is known — validate by plotting precision-recall across thresholds 0.2–0.6 and selecting the F1-optimal value):

- **Re-retrieval:** re-query with enriched prompt: original task + "current state: {VLM description of current frame}". If a higher-confidence memory is found, switch to it, resuming from the best-matching step.
- **New memory creation path:** if re-retrieval returns no memory above the confidence threshold (conf < 0.65; *calibration parameter* — requires a retrieval evaluation benchmark as described in Section 6.1 before the value can be validated), proceed with the task using the closest available memories as structural reference points for generalisation.

If the task completes successfully in the generalisation path: compile the full execution trajectory into a new `memory.md` using the same automated pipeline; add to the library with `confidence_level = 'auto_generated'` and `pending_review = true`; the memory is available for retrieval immediately but flagged for human review before inclusion in the next training cycle.

Negative Trajectory Structure Failed trajectories are stored in a dedicated `negative_trajectories/` subdirectory using the same schema. Each is tagged with a `failure_mode` field: `action_error`, `navigation_failure`, `context_loss`, or `task_abandonment`.

Use in RL Training Failure trajectories are incorporated into GRPO groups as forced low-reward members. Up to two failure trajectories per task are included alongside the $G = 8$ policy-sampled trajectories, with rewards clipped to $R_{\text{floor}} = -0.2$. The negative member count is capped at two per group.

Pre-Training Use A subset of failure trajectories appears in the pre-training corpus at a maximum ratio of 5% of the actuation command file slice. Failure trajectories are never included in the `memory.md` compiled document slice.

3.4.5 Two-Stage Retrieval Stack

Figure 9 — Two-Stage Retrieval Stack: Bi-encoder ANN + Cross-encoder Re-ranker

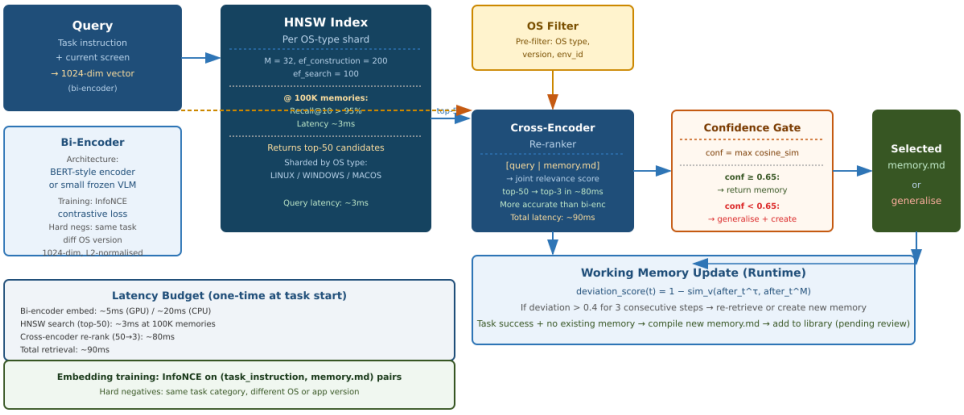

Figure 9: Two-Stage Retrieval Stack. Bi-encoder HNSW search generates top-50 candidates in ≈3ms. Cross-encoder re-ranks to top-3 in ≈80ms. Total retrieval budget ≈90ms. Confidence gate at 0.65 threshold.

3.4.6 Memory Staleness and Version Control

The Redis session registry stores `os_type`, `os_version`, and `os_environment_id` for every session. A staleness penalty is applied in the re-ranker:

$$\text{staleness}(m) = \max\left(0,\ \frac{\text{current\_date} - \text{memory\_created\_date}}{\text{staleness\_period}(m.\text{app\_category})}\right)$$

The staleness period is *not* uniform across applications — a uniform 180-day threshold is inappropriate because UI change rates differ substantially by application category. Recommended starting values (each requiring calibration from observed UI change rates in the Memory Archive corpus):

- Web applications: 30 days (frequent layout and navigation changes).
- Desktop productivity applications (office suites, creative tools): 90 days.
- OS shell tools and command-line interfaces: 365 days (rare interface changes).

These thresholds are *calibration parameters*, not system constants. Derive final values from the UI change rates observed in the collected corpus and recalibrate the confidence threshold (0.65) every time the library grows by 10K sessions.

3.4.7 Inference-Time Action Output Format

At each step during inference, the model generates a sequence bounded by the special tokens defined in Section 3.1.5. Three actuation artifacts serve distinct roles:

- **CommandEvent JSON** (delimited by `<|action|>`): the sole artifact consumed by ControlCenter. Extracted, deserialised, and forwarded via gRPC. Schema validation runs before dispatch; a validation failure triggers re-sampling.
- **Human-readable** `converted_action` (delimited by `<|converted_action|>`): retained in session context for working memory tracking and memory compilation. Not dispatched externally.
- **Reasoning text** (delimited by `<|reasoning|>`): used internally for deviation scoring and PRM quality assessment. Not dispatched externally.

The `cc_commands.json` replay format is not produced at inference time — CommandEvent JSON is dispatched directly.

3.4.8 Multi-Memory Retrieval and Context Construction

Trigger Condition Multi-memory retrieval is triggered when the top-1 cross-encoder score falls below 0.65 but two or more memories from the top-3 set together achieve combined task coverage > 0.75:

$$\text{coverage}(M_1, M_2) = \frac{|\text{entities(task)} \cap (\text{entities}(M_1) \cup \text{entities}(M_2))|}{|\text{entities(task)}|}$$

If neither single-memory nor multi-memory coverage exceeds threshold, the no-memory generalisation path (Section 3.4.4) is taken.

Context Construction When two memories are retrieved, both are included using paired `<|memory_start|>` / `<|memory_end|>` blocks. The higher-scoring memory is placed first.

```
<|memory_start|>
[Memory 1: higher cross-encoder score]
<|memory_end|>
<|memory_start|>
[Memory 2: lower cross-encoder score]
<|memory_end|>
[Step sequence begins]
```

At 32K context, two full memories consume approximately 40–50% of the context budget. Retrieval is capped at two memories.

Cross-Encoder Adaptation For multi-memory selection, a second cross-encoder pass scores each pair (M_i, M_j) from the top-3 jointly: $[\text{query}|M_i.\text{Overview}|M_j.\text{Overview}]$. The pair is selected if joint_score $>$ single_score $+ 0.05$.

Working Memory Under Multi-Memory Retrieval Deviation scoring (Section 3.4.4) operates per retrieved memory, computed against the memory whose step sequence most closely matches the current execution position. When execution transitions to a step region covered only by Memory 2, the deviation baseline switches automatically — no re-retrieval required.

4 Self-Generated Memory as In-Training Evaluation

A distinct advantage of the Memory Archive paradigm is that it provides an evaluation mechanism that does not require a separate benchmark or held-out test environment. At training checkpoints, the model being trained is run through live CUA sessions and asked to produce its own `memory.md`. Self-generated memory quality provides a multi-dimensional signal qualitatively different from benchmark evaluation — it measures internal reasoning and procedural structure, providing continuous, interpretable, and diagnostically rich signals available *during* training.

4.1 Overfitting Detection

A model that has overfit will produce self-generated memories that are verbatim reproductions of training examples. However, high similarity to training memories is *correct* behaviour for in-distribution tasks — a model that correctly executes a known task will naturally produce a memory close to the training memory for that same task. Conflating correct reproduction with overfitting produces false positives.

The overfitting signal requires an explicit in-distribution / out-of-distribution split:

- **In-distribution tasks**: tasks present in the training corpus. High self-generated memory similarity to training memories here is expected and correct.
- **Out-of-distribution tasks**: novel tasks held out from training entirely.

The overfitting criterion is: MinHash LSH similarity between a self-generated memory and the closest training memory > 0.85 on *OOD tasks only*. Additionally, track the generalisation gap:

$$G_{\text{gap}} = \overline{\text{sim}}(\text{in-distribution}) - \overline{\text{sim}}(\text{OOD})$$

A healthy model has $G_{\text{gap}} > 0$ (correctly reproduces known procedures with higher fidelity) but both values are meaningfully above zero. A $G_{\text{gap}} > 0.3$ with low absolute OOD similarity is the overfitting signal. Diagnostic: overfit steps on OOD tasks typically reproduce exact phrasing of training reasoning text rather than generating contextually appropriate descriptions of what was visible in the at-frame and after-frame.

4.2 Underfitting Detection

A model that has underfit will produce shallow reasoning, missing step sections, or reasoning entries that do not connect causally to adjacent steps. Measurement: step count completeness and reasoning depth score (token count per entry + causal connective density: presence of "because", "so that", "in order to", "which causes"). Signal interpretation: low reasoning depth early in training is expected; if it does not increase as training progresses, the reasoning loss weight may be insufficient or VLM annotation quality too low. Diagnostic: examine whether step headers are correctly numbered and ordered.

4.3 Context-Awareness Signal

Context-awareness is the most diagnostically valuable signal. A context-aware model's reasoning entries reference specific UI elements, values, and states visible in both the at-frame and the after-frame.

At-Frame Context The at-frame for mouse events carries the annotated click position. A context-aware model's reasoning should reference the specific element at those coordinates. A context-blind model says only "clicked the submit button" without spatial grounding.

After-Frame Context The after-frame is where the consequence of the action is visible. A context-aware model's reasoning must reference this outcome. A model that only references the at-frame is reasoning about what it did rather than whether it worked.

Measurement: entity extraction from reasoning text vs entity extraction from at-frame and after-frame. Target: > 0.75 average entity overlap over steps. Signal interpretation: low at-frame overlap indicates the model is not grounding spatial reasoning; low after-frame overlap indicates the model is not tracking visual outcomes.

4.4 Human-Level and Super-Human Signal

When a self-generated memory describes a shorter, more efficient procedure than the human-annotated baseline — fewer steps, better edge case coverage, more precise prerequisites, or a step sequence that avoids a documented failure mode — the model has found a superior execution path. A shorter memory alone is not the signal: a model that truncates memories early or elides required steps will also produce a lower step count while performing worse.

A self-generated memory is flagged as potentially super-human when all three conditions hold simultaneously:

1. **Step count ratio** < 1.0: self-generated step count is fewer than the human-annotated baseline.
2. $R_{\textbf{outcome}} > 0.85$: the shorter procedure achieves the same visual outcome as the human procedure (measured against the human memory's after-frames).
3. **Procedure completeness**: all prerequisite state transitions identified in the human memory's Prerequisites section are present in the self-generated memory — no required intermediate state is skipped.

Super-human memories that meet all three conditions are flagged for mandatory human review, and replace the corresponding human-annotated memory in the library if accepted. Memories with step count ratio < 1.0 but $R_{\text{outcome}} \leq 0.85$ are classified as step elision failures, not super-human performance.

4.5 Connection to the Self-Reinforcing Training Cycle

Self-generated memory evaluation actively feeds the training cycle. Memories that pass the quality checks in Sections 4.1–4.4 enter the memory library, enriching the pre-training corpus for the next training cycle. The ratio of accepted to flagged self-generated memories over training time is a single aggregate metric for training progress. This closes the self-reinforcing cycle in Figure 10: new memories created at inference (Section 3.4.4) and self-generated memories that pass quality review both flow back into the pre-training corpus.

5 Positioning in the Research Landscape

5.1 The Format Consistency Hypothesis

> ***Hypothesis:*** *A CUA model trained end-to-end with* `memory.md` *as the shared representational format — present at pre-training, SFT, post-training, and inference — will exhibit lower task distribution shift, better generalisation to novel environments, and more reliable execution of known tasks compared to a CUA trained without this format consistency, holding architecture and total compute constant.*

Experimental Protocol

- **Condition A** (full paradigm): pre-train on `memory.md` corpus → memory-conditioned SFT with actuation targets → memory adherence RL → retrieval at inference.
- **Condition B** (standard): same base model, no `memory.md` pre-training, standard (screenshot, action) SFT, outcome RL, zero-shot inference.

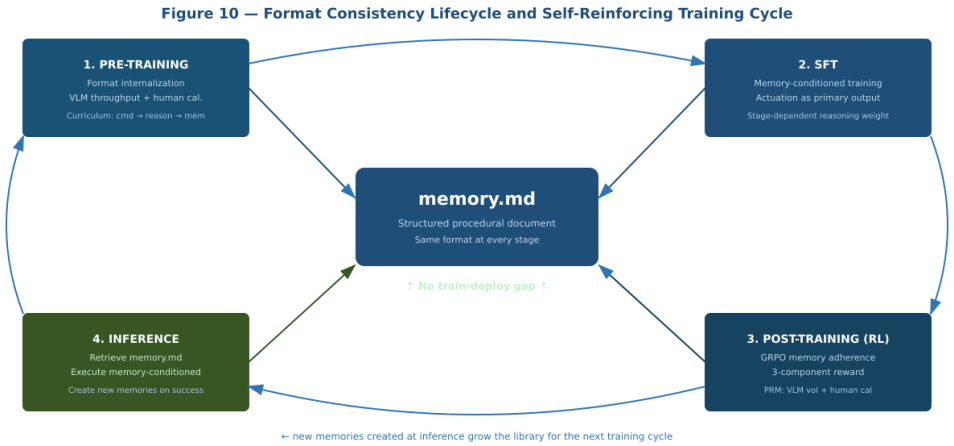

Figure 10: Format Consistency Lifecycle. `memory.md` at the centre of all four stages. New memories created at inference and self-generated memories passing quality review both feed back into the pre-training corpus, growing the library endogenously each training cycle.

- **Condition C** (retrieval only): same training as B, memory retrieval added at inference only.

Expected result: A > C > B. Condition C vs B tests inference-only retrieval value. Condition A vs C tests the additional value of format-consistent training.

Falsification Conditions and Statistical Power Three null hypotheses:

- $H_{0,\text{primary}}$: $\mu_A \leq \mu_B$. The full paradigm produces task success rates no higher than standard behavioral cloning with outcome RL.
- $H_{0,\text{retrieval}}$: $\mu_C \leq \mu_B$. Inference-time retrieval produces task success rates no higher than zero-shot execution.
- $H_{0,\text{training}}$: $\mu_A \leq \mu_C$. Format-consistent training produces task success rates no higher than retrieval-only.

Minimum Effect Sizes OSWorld [10] evaluation exhibits high variance ($\sigma \approx 0.18$–0.22 across runs). Effect sizes below 7–8 percentage points are within the noise floor. A vs B: $\Delta \geq 15$ pp; C vs B: $\Delta \geq 8$ pp; A vs C: $\Delta \geq 7$ pp (critical test of the Format Consistency Hypothesis).

Statistical Test Specification Use a one-tailed Wilcoxon signed-rank test at $\alpha_{\text{adj}} = 0.05/3 \approx 0.017$ per comparison (Bonferroni correction for three simultaneous null hypotheses). Sample size is computed per comparison at $\sigma = 0.20$, power $1 - \beta = 0.80$:

$$N = \frac{(z_{\alpha/k} + z_\beta)^2 \cdot \sigma^2}{\Delta^2}$$

- A vs B ($\Delta = 0.15$): $N \approx 11$ — trivially achievable.
- C vs B ($\Delta = 0.08$): $N \approx 39$.
- A vs C ($\Delta = 0.07$, critical): $N \approx 51$.

Use $N = 60$ evaluation tasks (the justified minimum for the hardest comparison), drawn via stratified sampling across OSWorld task categories and difficulty tiers — the task distribution is non-uniform and unweighted sampling would oversample near-zero-success-rate tasks that provide no discriminative signal. Report 95% confidence intervals on task success rate for each condition.

Partial Falsification Outcomes Four distinct outcomes warrant distinct interpretations: (1) A > C > B (all $p < 0.05$): hypothesis confirmed; (2) A > B but A ≈ C: format-consistent training adds no

value beyond retrieval; (3) C $>$ B but A $\approx$ C: same conclusion; (4) A $>$ B but C $\approx$ B: training helps without retrieval, but retrieval alone does not — surprising. Three confounds must be controlled: architecture parity, compute parity, and library independence.

5.2 Comparison with Existing CUA Training Approaches

Table 5 positions Memory Archive against five representative CUA systems across four axes: training data composition, process supervision availability, memory at inference, and format consistency. The "Format Consistency" column measures the fraction of training artifacts that appear structurally unchanged at inference time — a High rating means the model encounters the same representational object at training and deployment; Low means the two contexts differ substantially.

Behavioral Cloning and UI-TARS Standard behavioral cloning trains on (screenshot, action) pairs with no process supervision and deploys with a plain-text prompt, producing a Low format consistency rating — the training format and inference format are categorically different objects. UI-TARS [6] improves on this with synthetic chain-of-thought reasoning during training, yielding a Medium rating: reasoning is present at training but is not a structured artifact the model retrieves and follows at inference.

ICAL ICAL [3] retrieves experience examples at inference time, structurally similar to Memory Archive's retrieval. The key distinction: ICAL retrieves abstracted programs — compressed procedural summaries without per-step visual grounding or actuation command records. Memory Archive retrieves annotated visual trajectories with at-frame image references and the CommandEvent schema embedded in the memory. The training-inference format correspondence is tighter in Memory Archive because the `memory.md` structure — including at-frame references — is present during training as a first-class artifact. ICAL's High consistency rating is therefore an approximation; a more precise definition of the format consistency score as the fraction of training artifacts appearing unchanged at inference would place ICAL at Medium-High.

HyMEM HyMEM [2] shares the core commitment to persistent memory at inference, but uses graph-structured memory without per-step visual grounding. Memory Archive differs in two ways: (1) `memory.md` is a flat procedural document with at-frame image references per step, enabling spatial grounding through the R_{spatial} reward component; (2) the memory format is present during pre-training and SFT, not only at inference, giving Memory Archive's model an explicit prior over the structure it will retrieve.

SkillRL SkillRL [8] distils trajectory data into hierarchical skill representations used at inference. The training-to-inference gap is Medium: skills are extracted from trajectories but are structurally different from the raw trajectory format the model is trained on.

All "High/Medium/Low" ratings in Table 5 are qualitative assessments. A rigorous formal definition — fraction of training artifacts whose structure is preserved at inference, computed per-field — would refine these into continuous scores and is a necessary step before any direct claim of superiority over ICAL or HyMEM.

Table 5: Comparison of CUA training approaches.

System	Training Data	Process Labels	Memory at Inference	Format Consistency
Behavioral Cloning	(screenshot, action)	None	None	Low
UI-TARS / OpenCUA-32B [6]	Human demos + synthetic CoT	Synthetic	None	Medium
UI-R1 [9]	GUI trajectories + RL	None	None	Low
ICAL [3]	Abstracted trajectories via VLM	Implicit	Retrieved examples	High
HyMEM [2]	GUI trajectories	None	Graph-structured memory	Medium
SkillRL [8]	Trajectories $\rightarrow$ distilled skills	None	Hierarchical skills	Medium
Memory Archive	VLM/human: actuation+ reasoning+ vision+ `memory.md`	VLM-gen + human cal	`memory.md` (same as training)	High — all stages identical

6 Open Problems

6.1 Retrieval Accuracy (Critical)

Required: retrieval evaluation benchmark measuring precision@1, recall@5, and false positive rate, with task categories, OS environments, and application versions as filter axes. Build before deploying retrieval in any production session.

6.2 Memory Versioning and Staleness (High)

Implement `validity_env` metadata per memory, compatibility pre-filtering, and a staleness detector using R_{outcome} divergence to flag memories for re-annotation.

6.3 VLM Reasoning Quality and Human-VLM Consistency (High)

VLM reasoning hallucination, step conflation, and vocabulary drift degrade corpus quality silently. Required: human-VLM consistency metric over held-out sessions, per-step VLM annotation confidence score, and optionally a VLM-to-human alignment fine-tuning step.

6.4 Actuation Schema Drift (High)

If Control-Center's CommandEvent schema changes, all existing training data becomes partially misaligned. Required: schema version pinning in `metadata.json` and a migration pipeline for existing training data when the schema is updated.

6.5 Memory Composition (Medium)

Complex tasks spanning multiple memories require a composition mechanism. The design space has three distinct architectures:

Sequential Memory Chaining The retrieval pipeline is invoked multiple times during a single task execution. Requires no changes to the memory data structure or special token schema — it extends the working memory update logic in Section 3.4.4. The critical open question is what triggers the boundary between memories: a confidence drop in the running deviation score, an explicit end-of-procedure signal in the last step's Notes section, or step-count exhaustion relative to the retrieved memory's total step count.

Hierarchical Decomposition A high-level task memory references lower-level sub-procedure memories by name or identifier, structurally similar to SkillRL's [8] hierarchical skill representation. Implementing this architecture requires two schema additions: a References section in `memory.md` listing dependent memory IDs, and a `<|memory_ref|>` special token to mark cross-memory references within step text.

Cross-Linked Composition Individual step sections within one memory carry explicit typed links to steps in other memories that share a common pre-condition or post-condition (A-MEM-style [11]). The primary implementation challenge is that the HNSW index stores flat vectors without relational metadata — cross-linked composition requires a graph layer over the memory library absent from the current architecture.

Of the three, sequential chaining carries the lowest implementation risk. Hierarchical decomposition offers the most natural fit with complex multi-component CUA tasks. Cross-linked composition is best deferred until single-memory and hierarchical paths are validated.

6.6 Auto-Generated Memory Quality Gate (Medium)

Memories created via the new memory creation path (Section 3.4.4) enter the library with `pending_review = true`. The quality gate translates the Section 4 evaluation signals into a binary accept/reject decision.

Hard-Reject Pre-Filter Three hard-reject conditions apply: (1) Duplicate: MinHash LSH similarity to any existing library memory > 0.85; (2) Incomplete: $s_{\text{complete}} < 0.80$; (3) Context-blind: $s_{\text{context}} < 0.45$.

Table 6: Auto-Generated Memory Quality Gate: Scoring Components. Weights sum to 1.0 excluding the novelty gate. Entity extraction for s_{context} uses the same VLM annotation call as Section 4.3.

Signal	Metric	Formula / Measurement	Hard Gate	Weight
Novelty (hard gate)	MinHash LSH vs library	$\text{sim}(m, \mathcal{M}_{\text{library}})$, 128-permutation	REJECT if > 0.85	0.00
Structural completeness	Steps in memory / steps executed	$s_{\text{complete}} = \min(1, \lvert S_m \rvert / \lvert S_{\text{session}} \rvert)$	REJECT if < 0.80	0.25
Context grounding	Entity overlap: reasoning vs frames	$s_{\text{context}} = \lvert E_{\text{reason}} \cap E_{\text{frames}} \rvert / \lvert E_{\text{frames}} \rvert$	REJECT if < 0.45	0.35
Reasoning depth	Causal connective density + tokens/step	$s_{\text{depth}} = \min(1, \text{causal\_count} / (\lvert S \rvert / 3))$	None	0.25
Step ordering	Headers monotonically increasing and complete	$s_{\text{order}} \in \{0, 1\}$	None	0.15

Composite Score and Acceptance Thresholds

$$Q_{\text{auto}}(m) = 0.25 \cdot s_{\text{complete}} + 0.35 \cdot s_{\text{context}} + 0.25 \cdot s_{\text{depth}} + 0.15 \cdot s_{\text{order}}$$

Three acceptance tiers: **Accept** ($Q_{\text{auto}} \geq 0.70$ AND $s_{\text{context}} \geq 0.60$): enters training corpus with `pending_review = false`; **Pending** ($0.50 \leq Q_{\text{auto}} < 0.70$, or $Q_{\text{auto}} \geq 0.70$ but $s_{\text{context}} < 0.60$):

available for retrieval, enters training corpus only after lightweight human review; **Reject** ($Q_{\text{auto}} < 0.50$): retained in library as low-confidence candidate but does not enter training corpus.

The distribution of Q_{auto} scores over time is a continuous diagnostic for training health. A mean Q_{auto} that plateaus below 0.65 indicates the model has reached its current quality ceiling. Super-human memories (above acceptance threshold and step count ratio < 1.0) are flagged for mandatory human review regardless of Q_{auto}, and replace the corresponding human-annotated memory in the library if accepted.

6.7 Continual Learning and Memory Consolidation (Medium)

New sessions should refine existing memories rather than always producing new ones. The memory consolidation problem requires a merge operation preserving correct procedural sequence while incorporating newly discovered edge cases.

7 TurboQuant for Memory Efficiency at Inference

TurboQuant [12] is a training-free, model-agnostic online vector quantization algorithm that compresses KV cache vectors from 16-bit precision to approximately 3–3.5 bits per element, achieving near information-theoretic optimal inner-product preservation with zero quantization-constant memory overhead. It operates in two stages: PolarQuant applies a random orthogonal rotation to spread energy uniformly across dimensions; QJL (Quantized Johnson-Lindenstrauss) applies a 1-bit residual error correction that eliminates bias from attention score estimation. Empirically, 3.5-bit TurboQuant matches full-precision performance on LongBench and achieves approximately $8\times$ attention throughput on H100 at 4-bit precision relative to 32-bit keys.

This section covers two specific points where TurboQuant applies: inference KV cache compression (Section 7.1) and retrieval index compression (Section 7.2). It does not address model weight compression — TurboQuant is inference-only. QLoRA and TurboQuant are complementary: QLoRA targets weight memory footprint during training; TurboQuant targets the KV cache footprint during inference.

7.1 Inference KV Cache Compression

The Memory Archive inference pipeline produces a KV cache structure with two unusually demanding properties: (1) the context window grows continuously throughout execution as each step appends before_t, at_t, after_t image token encodings plus text tokens; (2) the retrieved `memory.md` document occupies the head of the context from step one — its KV entries are never evicted.

Memory Pressure Quantified For a 7B-class VLM backbone with head dimension $d = 128$ and $H = 32$ attention heads, a single KV entry costs $2 \times d \times 2$ bytes $= 512$ bytes at FP16. A 50-step session with 150 image tokens per image $\times$ 3 frames $= 450$ image tokens, plus ≈4,000 text tokens, produces 4,450 KV entries per head, or ≈70.7 MB of KV cache at FP16 across all 32 heads. Formulaically:

$$\text{KV size (FP16)} = 2 \times d \times H \times T \times 2 \text{ bytes}$$

$$\text{KV size (TurboQuant)} = 2 \times d \times H \times T \times \frac{b}{8} \text{ bytes}, \quad b \approx 3.5$$

At 3.5-bit TurboQuant the same session occupies ≈15.5 MB — a 4.6$\times$ reduction. A GPU serving four concurrent inference sessions at FP16 requires ≈283 MB for KV caches; at 3.5-bit, the same hardware can serve 18 concurrent sessions. Alternatively, TurboQuant extends the effective context window from 32K to approximately 128K–150K tokens within the same hardware budget.

The K/V Norm Ratio Constraint Empirically, modern VLMs exhibit substantially different Key and Value vector magnitudes. Practical findings from KV quantisation studies: K/V ratio $< 10\times$ — 3-bit uniform allocation works without degradation; K/V ratio 10–60$\times$ — requires asymmetric bit

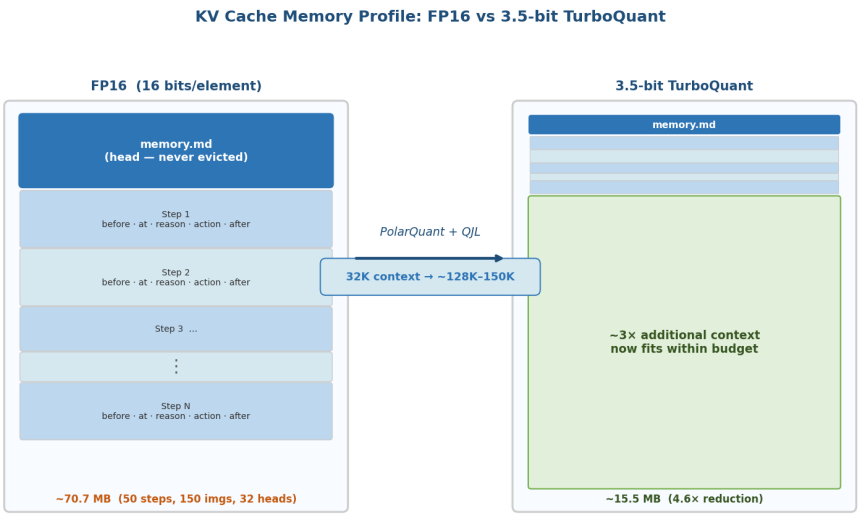

Figure 11: KV Cache Memory Profile: FP16 vs 3.5-bit TurboQuant. Left: FP16 session context with `memory.md` head (never evicted) and step-by-step accumulation. Right: TurboQuant-compressed equivalent at 4.6× reduction. A 32K context window extends to approximately 128K–150K tokens within the same hardware budget.

allocation, approximately 4.5–5 bits for Keys; K/V ratio $> 100\times$ — TurboQuant alone is insufficient. The K/V norm ratio must be measured empirically for the chosen backbone before committing to a uniform 3.5-bit allocation; the specific breakpoints above are indicative and may shift depending on the model architecture and fine-tuning history.

7.1.1 The At-Frame Visual Token Distribution — An Open Research Question

The Memory Archive inference context is different from standard LLM benchmarks: it interleaves three qualitatively distinct image types at regular intervals, and mouse at-frames carry a programmatically drawn click annotation (red circle, directional arrow, coordinate box) that concentrates image information in a spatially bounded region. When a ViT encodes this image, patch tokens covering the annotated region receive systematically higher activation norms than background patches. The resulting KV vectors for annotated patch tokens are drawn from a different distribution than unannotated patch tokens.

PolarQuant's random orthogonal rotation is data-oblivious. For unannotated image patches and text tokens, the approximately isotropic assumption is reasonable. For annotated mouse at-frame patch tokens, the assumption is less obviously satisfied: the spatial concentration of the annotation creates a non-isotropic component, and whether the rotation fully whitens this or whether annotated patch tokens survive as outliers is an empirical question.

If annotated patch tokens produce KV outliers that TurboQuant's rotation does not fully handle, the model's ability to ground spatial reasoning in the click position could degrade. Monitoring R_{spatial} distributions before and after TurboQuant activation provides the empirical test: a drop in mean R_{spatial} not accompanied by a corresponding drop in R_{align} or R_{outcome} isolates the at-frame compression pathway as the cause. This is the primary empirical investigation required before deploying TurboQuant on the inference KV cache of a production Memory Archive system.

7.2 Retrieval Index Compression

The two-stage retrieval stack (Section 3.4.5) builds an HNSW index over 1024-dimensional L2-normalised float32 embedding vectors, one per memory. At 100K memories this index occupies approximately 400 MB of raw embedding storage. At 1M memories — a realistic long-term target — raw embedding storage reaches approximately 4 GB, no longer fitting comfortably in GPU VRAM.

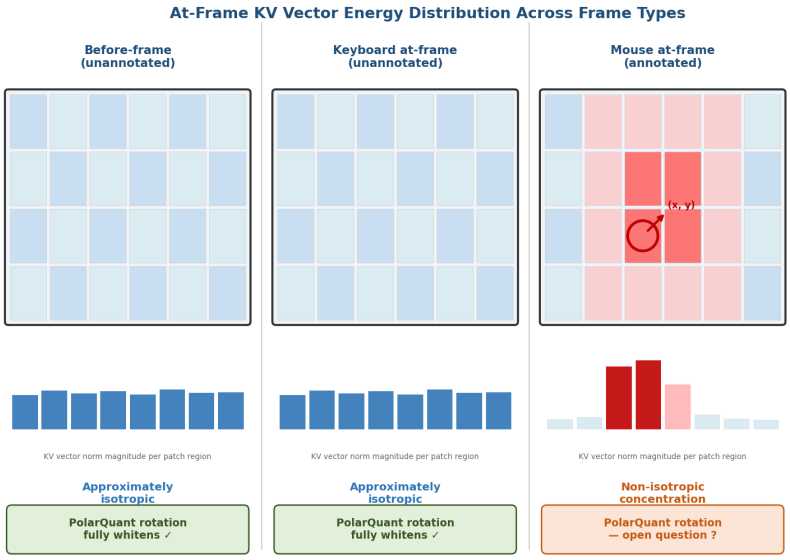

Figure 12: At-Frame KV Vector Energy Distribution Across Frame Types. Before-frames and keyboard at-frames (left, centre) exhibit approximately isotropic energy distributions — PolarQuant's rotation fully whitens these. Mouse at-frames (right) carry a programmatically drawn click annotation that concentrates energy in a spatially bounded region, producing non-isotropic patch token distributions.

TurboQuant's QJL component is designed specifically for high-recall approximate nearest-neighbour search over inner products on L2-normalised vectors. The Memory Archive bi-encoder produces 1024-dim vectors — a lower-dimensional regime than TurboQuant's benchmarks, making compression strictly easier. The recall@10 $> 95\%$ target already specified in Section 3.4.2 is directly comparable to TurboQuant's published nearest-neighbour recall results.

The compression effect on index size:

$$\text{Index size (FP32)} \approx N_m \times 1024 \times 4 \text{ bytes}$$

$$\text{Index size (3.5-bit TurboQuant)} \approx N_m \times 1024 \times 0.4375 \text{ bytes}$$

giving a $4.6\times$ compression factor:

- 100K memories: FP32 $\approx$ 400 MB $\rightarrow$ 3.5-bit TurboQuant $\approx$ 87 MB
- 1M memories: FP32 $\approx$ 4.0 GB $\rightarrow$ 3.5-bit TurboQuant $\approx$ 870 MB

At 1M memories, the TurboQuant-compressed index fits comfortably in 24 GB GPU VRAM alongside the model weights, eliminating host-to-device transfer latency. The HNSW graph structure overhead (approximately $2\times$ the raw embedding size for $M = 32$) applies to both variants and is not affected by TurboQuant.

Incremental Addition Compatibility TurboQuant's quantisation is data-oblivious — it derives quantisation parameters analytically from vector dimension and target bit width, requiring no calibration dataset. Each new memory's embedding vector is quantised at insertion time using the same fixed rotation and codebook, making TurboQuant-compressed HNSW insertion a drop-in replacement for FP32 insertion.

Cross-Encoder Compatibility TurboQuant applies only to the bi-encoder HNSW index that generates the top-50 candidate set. The cross-encoder re-ranker (Section 3.4.3) operates over full

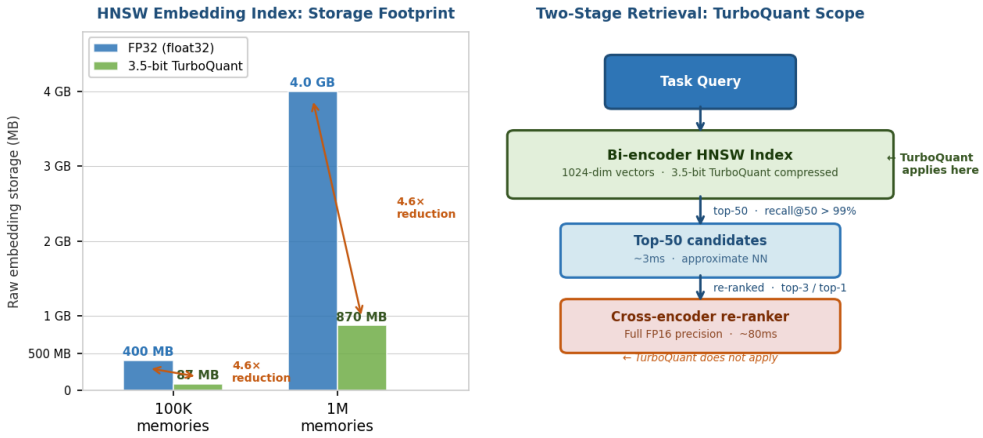

Figure 13: Retrieval Index Compression at Scale. Left: raw embedding storage footprint at 100K and 1M memories for FP32 vs 3.5-bit TurboQuant, showing $4.6\times$ reduction. Right: two-stage retrieval pipeline with TurboQuant scope marked — compression applies only to the bi-encoder HNSW index (Stage 1); the cross-encoder re-ranker (Stage 2) is unaffected.

`memory.md` text and is not affected. Any marginal recall degradation from compression at the bi-encoder stage is subsequently corrected by the cross-encoder's full-precision re-ranking.

8 Limitations and Scope

> ***Important:*** *The Memory Archive training paradigm presented in this document has not been trained end-to-end at the time of writing. All numerical hyperparameters — reward weights ($\alpha = 0.3$, $\beta = 0.4$, $\gamma = 0.3$), GRPO group size ($G = 8$), LoRA rank and learning rates, token loss weights, data mixing ratios, curriculum phase splits, PRM calibration weights, retrieval thresholds ($conf = 0.65$, $dev = 0.4$), and the staleness period — are* design recommendations *derived from principled reasoning and analogous published systems. They have not been empirically validated on any Memory Archive training run and should be treated as starting points requiring held-out validation before committing to production use.*

The next validation milestone is a small-scale pre-training experiment on a 1B–3B parameter VLM that produces at least one perplexity curve and one ScreenSpot accuracy datapoint against the hyperparameters recommended here. The experimental protocol in Section 5.1 constitutes the proposed evaluation plan for that empirical validation. The core architectural contributions — the artifact taxonomy, format consistency lifecycle, three-component reward, and retrieval stack — are independent of this empirical gap.

9 Summary

The Memory Archive training paradigm's primary contributions are: (1) format consistency across all four lifecycle stages; (2) actuation artifacts as first-class training targets teaching the model the CommandEvent schema and replay format; (3) process supervision at VLM scale; (4) a three-component memory adherence reward providing dense structured RL signal; (5) a two-stage retrieval stack with working memory updates and new memory creation on task success; and (6) self-generated memory as an in-training evaluation mechanism providing multi-dimensional diagnostic signals without requiring a separate benchmark environment. The first two engineering investments following cloud storage completion should be the retrieval evaluation benchmark (Section 6.1) and VLM reasoning consistency monitoring (Section 6.3).

Table 7: Memory Archive training paradigm: stage-by-stage summary.

Stage	Primary Artifacts	Algorithm	Key Parameters	Novelty
Pre-training	`memory.md` + `reasoning.jsonl` + actuation files + vision	Autoregressive LM, 3-phase curriculum	Mix 40/30/20/10. Phases 20/40/40%.	Format + actuation schema internalization at VLM scale
SFT	All MA artifacts + retrieved `memory.md`	Formulation B memory-conditioned, per-token weighted loss	$w_{\text{action}} = 1.0$, $w_{\text{header}} = 1.0$, $w_{\text{reason}} = 0.75 \rightarrow 0.5$, $w_{\text{memory}} = 0.0$	Actuation as primary target. Stage-dependent reasoning weight.
Post-training	`reasoning.jsonl` + `memory.md` adherence reward	GRPO, 3-component reward, adaptive KL	$G = 8$, $\alpha = 0.3$, $\beta = 0.4$, $\gamma = 0.3$, $\tau_{\text{px}} = 50/10$px	Memory adherence reward. Mixed-source PRM. Parallel OS instances required for rollouts.
Inference	`memory.md` library	2-stage retrieval + working memory update + new memory creation	conf_thresh=0.65, dev_thresh=0.4 (3 steps)	New memory creation on success. Same format as training.
Evaluation	Self-generated memories	Quality scoring vs baseline: overfit/ underfit/context/ super-human	Similarity thresh 0.85, entity overlap 0.75	In-training evaluation without external benchmark.

Acknowledgments

I would like to acknowledge Anthropic's Claude for its assistance during the research and conceptualisation phase of this work. Additionally, both Claude and Google's Gemini provided invaluable support in debugging, formatting, and typesetting the final LaTeX manuscript.

References

[1] Bonatti et al. WindowsAgentArena: Evaluating Multi-Modal OS Agents at Scale. arXiv:2409.08264, 2024.

[2] HyMEM. Hybrid Self-evolving Structured Memory for GUI Agents. arXiv:2603.10291, 2025.

[3] Sarch et al. (ICAL). VLM Agents Generate Their Own Memories: Distilling Experience into Embodied Programs of Thought. arXiv:2406.14596, 2024.

[4] Lightman et al. Let's Verify Step by Step. OpenAI / ICLR 2024, 2023.

[5] Luo et al. Improve Mathematical Reasoning via Automated Process Supervision. arXiv:2406.06592, 2024.

[6] Qin et al. UI-TARS: Pioneering Automated GUI Interaction with Native Agents. arXiv:2501.12326, 2025.

[7] Shao et al. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. arXiv:2402.03300, 2024.

[8] SkillRL. Evolving Agents via Recursive Skill-Augmented Reinforcement Learning. arXiv:2602.08234, 2025.

[9] UI-R1. Enhancing GUI Agent Reasoning with Action-Focused Reinforcement Learning. 2025.

[10] Xie et al. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. arXiv:2404.07972, 2024.

[11] Xu et al. A-MEM: Agentic Memory for LLM Agents. arXiv:2502.12110, 2025.

[12] Zandieh et al. TurboQuant: Online Vector Quantization with Near-optimal Distortion Rate. Google Research / NYU / Google DeepMind. ICLR 2026. arXiv:2504.19874, 2026.